

# Evaluating Cryptographic API Misuse Detectors for Go

Vivi Andersson, Martin Monperrus

KTH Royal Institute of Technology, Stockholm, Sweden

April 18, 2026  
Rio de Janeiro, Brazil  
SVM 2026

# A Cryptographic API Misuse

## Traefik, CVE-2025-66491. ingress-nginx provider:

```
InsecureSkipVerify: strings.ToLower(  
    ptr.Deref(cfg.ProxySSLVerify, "off")) ==  
    "on",
```

- The user sets proxy-ssl-verify: "on", expecting TLS verification to be **enabled**.
- Expression evaluates to true, InsecureSkipVerify becomes true, inverting security assumptions.
- TLS encryption remains, but backend authentication is disabled: Traefik may trust a spoofed backend.

# Why Go

- Go runs critical infrastructure: Kubernetes, Terraform, Traefik, Let's Encrypt's Boulder.
- Cryptographic misuses reach production, for example CVE-2025-66491 in Traefik and CVE-2024-45337 in [golang.org/x/ssh](https://go.dev/issue/60000).
- Four Go detectors exist (GOPHER, CODEQL, GOSEC, SNYK CODE), but none have been systematically compared.

## This paper

The first comparative study of cryptographic API misuse detectors for Go.

# Research Questions

- **RQ1.** What is the design space of state-of-the-art tools for detecting cryptographic API misuses in Go?
  - ▶ We survey the tools and consolidate a taxonomy of 14 Go crypto misuse classes.
- **RQ2.** To what extent are crypto misuse detection tools for Go consistent with each other?
  - ▶ We run all four tools on 328 security-relevant Go projects and compare their findings.

- **CodeQL** (GitHub, open source). Datalog queries over a fact database, with data-flow reasoning.
- **Gopher** (academic, CCS '24). Taint tracking on SSA, with dynamically updatable rules.
- **Gosec** (community, open source). SSA for some rules, AST pattern matching for others.
- **Snyk Code** (commercial). Proprietary static analysis; implementation details undisclosed.

# RQ1: Taxonomy and Tool Coverage

| Category   | #  | Rule                     | CodeQL | Gopher | Gosec | Snyk |
|------------|----|--------------------------|--------|--------|-------|------|
| Primitives | 01 | Insecure algorithms      |        | ✓      | ✓     | ✓    |
|            | 02 | Insecure PRNG            | ✓      | ✓      | ✓     | ✓    |
|            | 03 | Deprecated Go function   |        | ✓      |       |      |
| Key mgmt   | 04 | Constant/predictable key |        | ✓      |       |      |
|            | 05 | Short key length         | ✓      | ✓      | ✓     | ✓    |
|            | 06 | Static/predictable IV    |        | ✓      | ✓     |      |
| PBKDF      | 07 | Short salt length        |        | ✓      |       |      |
|            | 08 | Predictable salt         |        | ✓      |       |      |
|            | 09 | Low hash iterations      |        | ✓      |       |      |
| Transport  | 10 | HTTP protocol            |        | ✓      |       |      |
|            | 11 | TLS/SSL issues           | ✓      | ✓      | ✓     | ✓    |
| SSH        | 12 | Insecure SSH suite       |        | ✓      |       |      |
|            | 13 | No host key validation   | ✓      | ✓      | ✓     |      |
| Token      | 14 | No JWT verification      | ✓      |        |       |      |

Only 3 of 14 rules (insecure PRNG, short key, TLS/SSL) are covered by all four tools.

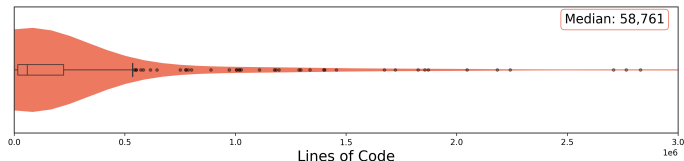
# RQ1: Taxonomy and Tool Coverage

| Category   | #         | Rule                          | CodeQL | Gopher | Gosec | Snyk |
|------------|-----------|-------------------------------|--------|--------|-------|------|
| Primitives | 01        | Insecure algorithms           |        | ✓      | ✓     | ✓    |
|            | 02        | Insecure PRNG                 | ✓      | ✓      | ✓     | ✓    |
|            | 03        | Deprecated Go function        |        | ✓      |       |      |
| Key mgmt   | 04        | Constant/predictable key      |        | ✓      |       |      |
|            | <b>05</b> | <b>Short key length</b>       | ✓      | ✓      | ✓     | ✓    |
|            | 06        | Static/predictable IV         |        | ✓      | ✓     |      |
| PBKDF      | 07        | Short salt length             |        | ✓      |       |      |
|            | 08        | Predictable salt              |        | ✓      |       |      |
|            | 09        | Low hash iterations           |        | ✓      |       |      |
| Transport  | 10        | HTTP protocol                 |        | ✓      |       |      |
|            | <b>11</b> | <b>TLS/SSL issues</b>         | ✓      | ✓      | ✓     | ✓    |
| SSH        | 12        | Insecure SSH suite            |        | ✓      |       |      |
|            | <b>13</b> | <b>No host key validation</b> | ✓      | ✓      | ✓     |      |
| Token      | 14        | No JWT verification           | ✓      |        |       |      |

**#05** all four tools support it. **#11** the largest class by detection volume. **#13** rare enough for full manual analysis.

## RQ2: Dataset

- **328 open-source Go repositories**, filtered from a curated set of 920 popular projects.
- Kept only actively maintained projects (commit within the past year, stable release) where cryptography is core.
- Size is right-skewed: median 58,761 LOC.





## RQ2: Execution Success and Detection

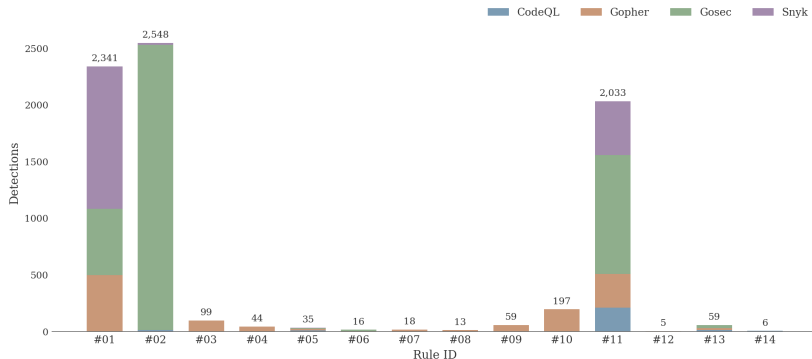
|                                      | CodeQL | Gopher | Gosec | Snyk |
|--------------------------------------|--------|--------|-------|------|
| <b>Projects analyzed (%)</b>         | 91.8   | 77.1   | 100   | 100  |
| <b>With findings (% of analyzed)</b> | 30.9   | 64.8   | 84.5  | 92.7 |
| <b>With findings (% of dataset)</b>  | 28.4   | 50.0   | 84.5  | 92.7 |

### Coverage

SNYK CODE and GOSSEC analyze all 328 projects, while GOPHER fails on 75 (runtime errors) and CODEQL on 27 (database build errors).

## RQ2: Detection Magnitude

- 7,473 total detections across 328 projects
- GOPHER uniquely detects issues for 6/14 rules (#03<sub>u</sub>, #04, #07-#10<sub>u</sub>, and #12<sub>u</sub>)
- Excessive detections for low-severity rules (e.g. #02 Insecure PRNG) esp. from Gosec

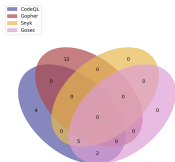


## RQ2: Detection Agreement

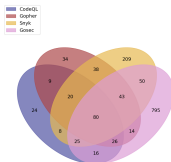
- 7,473 detections across 328 projects, collapsing to 6,152 unique locations.
- Only **1.3% of detections** are shared by all four tools.
- 92.6% of GOSEC detections and 63.5% of SNYK CODE detections are unique to that tool.

## RQ2: Agreement Depends on the Misuse Rule

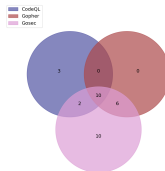
- Rule #05: all four tools support it, yet overlap remains limited.
- Rule #11: the largest class, with 2,033 detections total and 1,049 from GOSec alone.
- Rule #13: only 59 detections total, making manual inspection feasible.



Rule 05



Rule 11



Rule 13

### RQ2 takeaway

Agreement depends strongly on the misuse class.

# Manual Analysis: When Does Agreement Mean Truth?

- **Rule #05.** 4 of 5 detections shared by three tools (missed by GOPHER) are in mock or example code. Overlap is not always a correctness signal.
- **Rule #11.** 3 of 5 sampled GOSEC-only detections are false positives. SNYK CODE-only sample: no clear false positives.
- **Rule #13.** All 5 detections shared across tools are true positives. CODEQL uniquely catches custom callbacks beyond known patterns.

# What Goes Wrong: Three Examples

## 1. Same call, different meaning

*Rule #05, flagged by GOPHER*

```
h := blake2b.New512(nil) // bug if used as a MAC
                          // fine for content hashing
```

Static analysis cannot tell which one this is.

## 2. Pattern match misses the equivalent

*Rule #13, only CODEQL*

```
cfg.HostKeyCallback = func(...) error { return nil }
```

CODEQL detects this pattern; the other tools do not report it in our study.

## 3. Field name flagged, value ignored

*Rule #11, GOSEC false positive*

```
tls.Config{MinVersion: tls.VersionTLS12} // safe
```

In our manual sample, GOSEC flags this pattern even when the configured value is TLS 1.2.

# Implications

- **For practitioners:** no single detector is enough. The tools miss different things, so relying on one leaves systematic blind spots.
- **For evaluators:** raw counts mislead. On insecure PRNG detection, GOSSEC reports 2,517 findings where GOPHER reports 0, on the same code and for the same rule.
- **For tool builders:** the gap is qualitative, not only quantitative. The tools differ in rule coverage and in how much program context they model.

## Closing claim

Detector choice changes both what practitioners see in code review and what researchers conclude in evaluation.

- **API-level security invariants.** The paper points to formalization and enforcement of API-level security invariants as a research need.
- **More precise static reasoning.** Several disagreements come from limited context sensitivity around how values are configured and used.



Thank you

Questions and discussion